Physical Design Report

RISC-V Pipelined CPU

From RTL to a Routed Die on a 45 nm Standard Cell Library

Design Author: Elliot Staresinic

Design: pipelined_cpu_core, five-stage RISC-V, RV32I subset

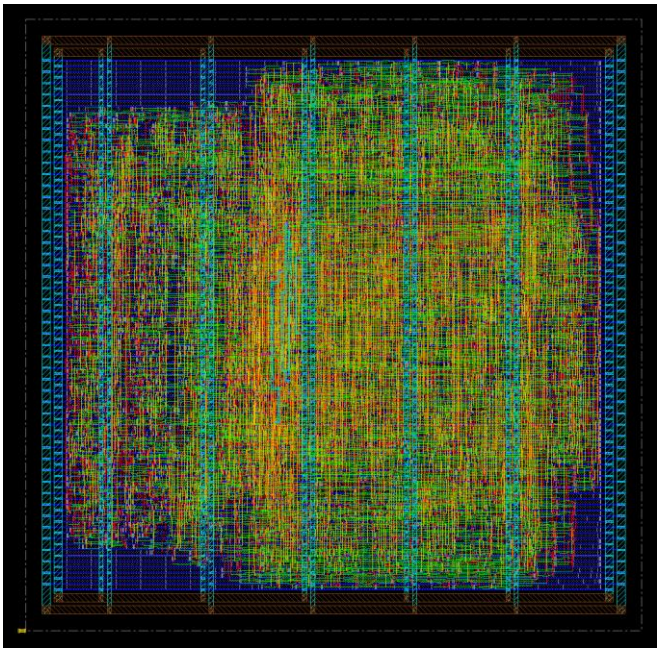
Run: 00-baseline | Clock: 3.00 ns (333 MHz)

Library: Nangate45, typical corner, QRC extraction

Tools: Cadence Genus and Innovus 23.12-s091_1

Date: Sat Aug 8 17:07:41 2026

TIMING RESULT: SETUP MET · 10 HOLD PATHS VIOLATED



Contents

1. Executive Summary

This report documents the physical implementation of a five-stage pipelined RISC-V CPU (pipelined_cpu_core), carried from RTL to a routed layout through Cadence Genus and Innovus on the Nangate45 45 nm standard cell library. Every number in this document is parsed directly from the run's own reports rather than transcribed.

The design routes to completion and meets setup timing at a 3.00 ns clock, with a worst negative slack of 0.009 ns on the critical path. It occupies 4,321 standard cells at 69.7 percent density, wired with 79,481 microns of copper.

Setup is met and hold is not. 10 hold paths are reported as violated, which is a constraint artefact rather than a physical one: this run modelled a single clock uncertainty for both checks, so the setup margin was subtracted from the hold check as well. Splitting the two uncertainties is the first change made in the next iteration.

Metric	Value	Source
Setup WNS	**0.009 ns**	40_final_setup.rpt
Setup paths violated	0	40_final_setup.rpt
Hold WNS	-0.014 ns	41_final_hold.rpt
Hold paths violated	10	41_final_hold.rpt
Standard cells	4,321	44_summary.rpt
Filler cells	7,236	44_summary.rpt
Flip-flops	1,347	44_summary.rpt
Logic area	11909.6 um2	44_summary.rpt
Core area	17094.0 um2	44_summary.rpt
Core density	69.7 %	44_summary.rpt
Total wire length	79,481 um	44_summary.rpt

Table 1: Signoff numbers, parsed directly from the run's own reports. Every figure in this document is read from the files named in the third column.

2. The Implementation Flow

Synthesis turns Verilog into a netlist: a flat list of logic gates and the wires between them, carrying no physical meaning whatsoever. Place and route turns that list into geometry, deciding where every gate physically sits and what copper connects them. The stages below run in order, and the design is written to disk after each one so a late failure never costs the earlier work.

2.1 Stages

Stage	Command	What it produces

Stage	Command	What it produces
1	floorPlan	A core box solved from a density target, not a fixed size
2	addRing, addStripe, sroute	A power grid, which the netlist never mentions
3	place_opt_design	Coordinates, plus logic restructuring to meet timing there
4	clock_opt_design	A real buffered clock tree, replacing the ideal clock
5	addFiller	Continuous power rails and wells across every row
6	route_opt_design	Copper, then re-optimisation against extracted parasitics
7	verify_drc	A check that the geometry obeys the routing rules
8	report, defOut	The reports this document is built from

Table 2: The place-and-route flow, stage by stage.

2.2 Why the Flow Is Written in Tcl

Innovus is, at its core, a Tcl interpreter. It is a Tcl shell with several thousand extra commands compiled in and a database of the chip attached to it. Typing place_opt_design at the Innovus prompt is typing Tcl, and running innovus -files scripts/innovus.tcl means nothing more than reading those same commands from a file instead of from the keyboard. There is no compilation step and no difference between the scripted tool and the interactive one.

This is not a Cadence quirk. Tcl was designed in the late 1980s specifically to be an embeddable command language for applications, EDA adopted it early, and it stayed. Cadence, Synopsys and Siemens tools are all driven this way, so the skill transfers across the industry in a way that familiarity with one vendor's graphical interface does not.

File	Language	Talks to	Job
run.sh	bash	Linux	Check prerequisites, make directories, launch tools
scripts/genus.tcl	Tcl	Genus	RTL to gate-level netlist
scripts/innovus.tcl	Tcl	Innovus	Netlist to routed layout
constraints/*.sdc	Tcl	any timer	Declare what correct timing means

Table 3: Three languages, three jobs. The shell script never mentions the chip; it confirms the tools and libraries exist and hands off.

Constraints are portable because they are Tcl. SDC is Tcl as well, and that is why it is portable. create_clock -name clk -period 3.0 is a procedure call, and because every vendor's timing engine implements the same procedure names, one constraint file feeds synthesis, place and route, and a signoff timer unchanged. That portability is why constraints live in their own file rather than inside the implementation script.

Why it is scripted at all. Physical design is not done in a graphical interface. The layout view is for inspecting a result, never for producing one. Production blocks run in batch on a compute farm, unattended, for hours or days, and a flow that cannot run headless cannot be run at scale. Scripting also makes a run reproducible: because every input is a text file under version control,

it is possible to say exactly what changed between two results, which is the entire basis of being able to iterate on one.

3. Timing

Slack is the margin on the worst path. Positive means the design works at that clock, negative means it does not, and the magnitude says by how much. Tracking it stage by stage is how you tell whether a timing problem lives in synthesis, in placement, in the clock tree or in the routing.

3.1 Slack Through the Flow

After	Clock model	WNS (ns)	What happened
Netlist as placed	ideal	-0.272	Starting point, before optimisation
place_opt_design	ideal	0.019	Placement optimisation
clock_opt_design	propagated	0.018	A real clock tree replaces the ideal one
route_opt_design	propagated + RC	**0.009**	Real copper, extracted parasitics

Table 4: Setup slack through the flow. The starting point is read from the optimiser's own progress table in reports/um/flow_QOR_summary.rpt.*

Reading the table. Placement did nearly all of the work. The netlist synthesis handed over was 272 picoseconds short the moment real coordinates existed, and placement optimisation recovered 291 picoseconds of that. Building the actual clock tree cost 1 picosecond and routing real wires cost a further 9 picoseconds.

3.2 Where the Time Actually Goes

The worst path launches from a register in MEM/WB, crosses the forwarding unit that decides EX must take a forwarded value rather than the register file's, and then ripples the carry across all 32 bits of the ALU adder before being captured in EX/MEM. Of the 2.863 ns that path takes, 2.123 ns is the carry chain alone: 74 percent of the clock period sits in one ripple-carry adder. That is the single change worth making to the design, and the adder is under 3 percent of the area, so speed there is cheap.

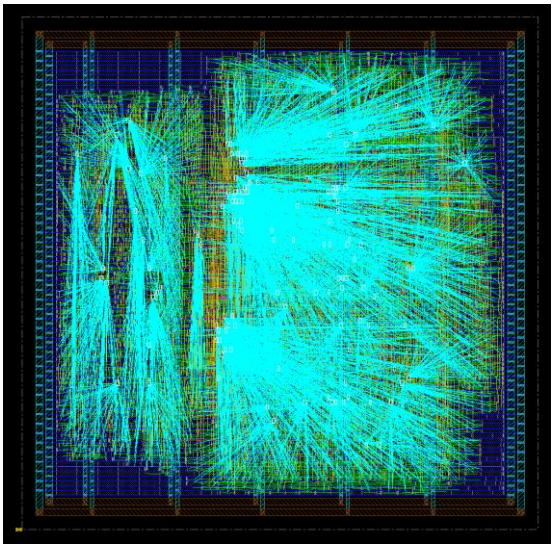


Figure 1: The synthesised clock tree driving every flip-flop in the design.

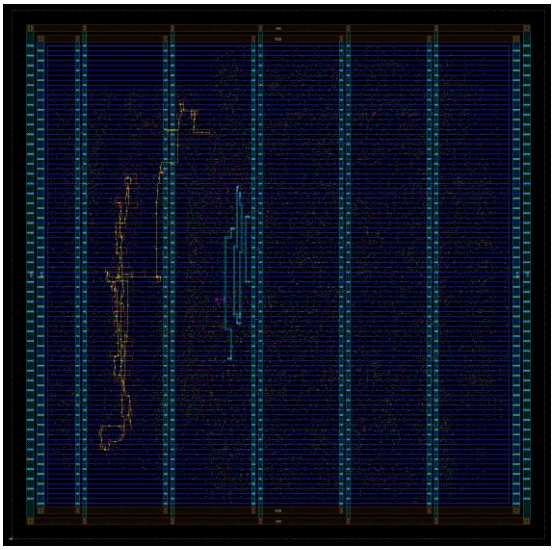


Figure 2: The critical path highlighted across the die. It launches in MEM/WB, crosses the forwarding logic and ripples through all 32 bits of the ALU adder before being captured in EX/MEM.

3.3 What This Number Does Not Tell You

A run that meets timing does not tell you what was achievable. The optimiser converges to whatever constraint it was given and then stops, and its area reclaiming passes deliberately spend positive slack to recover area and leakage. Synthesis stops trying at the same constraint. Finding the real maximum frequency means tightening the period and rerunning both tools until it fails.

4. The Physical Result

4.1 Cells and Fillers

The core holds 4,321 standard cells of real logic alongside 7,236 filler cells. Fillers contain no logic at all; standard cells carry power rails and wells that must be physically continuous across a row, and fillers exist purely to bridge the gaps left between placed cells. Without them the rails break and the design is not manufacturable.

4.2 Routing by Metal Layer

Layer	Wire length (um)	Share
metal1	0	0.0 %
metal2	23,035	29.0 %
metal3	30,336	38.2 %

Layer	Wire length (um)	Share
metal4	15,137	19.0 %
metal5	5,257	6.6 %
metal6	5,575	7.0 %
metal7	6	0.0 %
metal8	135	0.2 %
metal9	0	0.0 %
metal10	0	0.0 %

Table 5: Routing by metal layer. metal1 carries no signal because it is reserved for wiring inside the cells and for the power rails. The upper layers are nearly empty because a block this small has no journeys long enough to need them.

4.3 The Power Grid

Power is added, never inherited. The power grid is worth seeing on its own, because none of it comes from the netlist. A netlist describes logic and says nothing about how current reaches it. Every ring, strap and rail below was created by three commands in the flow script, and the connection between a cell's power pin and the VDD net exists only because globalNetConnect declared it.

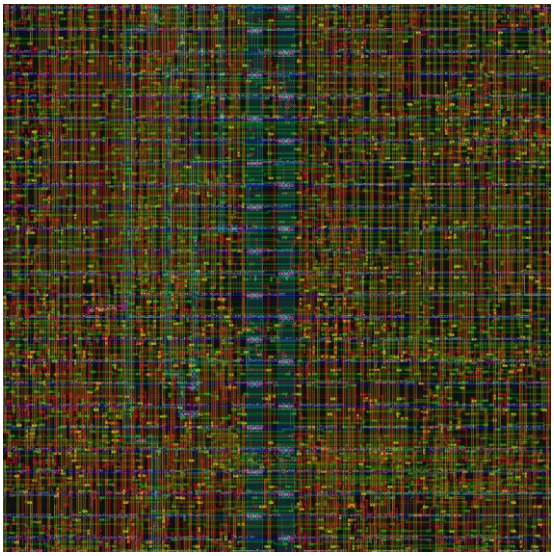


Figure 3: Detail. Individual standard cells sitting in their rows, with the metal1 power rails running along each row boundary.

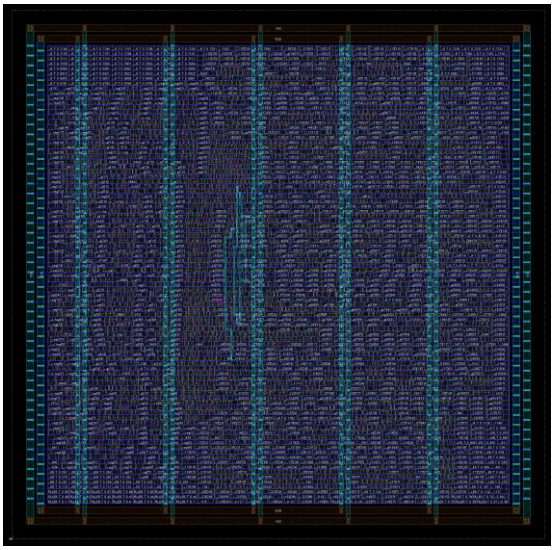


Figure 4: The power grid with signal routing hidden. The ring runs on metal9 and metal8 around the core, vertical straps cross on metal8 every 25 microns, and sroute ties the whole grid down to the metal1 rails running inside every standard cell row.

4.4 Where the Area Actually Goes

Of the 1,347 flip-flops, 1,023 are SDFF, a cell with a 2x1 multiplexer built into it, which is what an enable-guarded register maps onto. That count is the register file exactly: 31 architectural

registers of 32 bits, x0 costing nothing because it is hardwired to zero. Those cells alone are 6275 um2, or 53 percent of all logic area, and the multiplexer tree that reads them adds more. The structure that dominates the area and the structure that dominates the clock are different parts of the design.

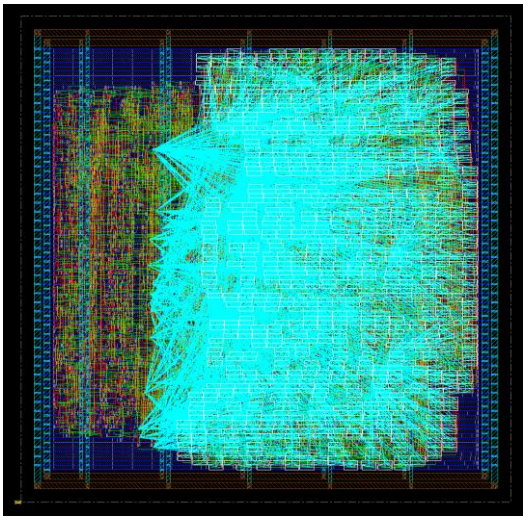


Figure 5: The register file selected across the die. All 992 flip-flops matching datapath/registers_RF_reg, which is 31 architectural registers of 32 bits each, x0 costing nothing because it is hardwired to zero.

5. What a Production Flow Has That This One Does Not

The structure of this flow matches industry practice. The rigour does not, in ways worth naming explicitly.

- **One corner is not signoff.** This run used a single library, a single RC corner and a single analysis view. Real signoff is multi-corner multi-mode, spanning slow and fast silicon, hot and cold, high and low voltage, and functional against test modes. The same library was also used for setup and hold, where production uses a slow corner for setup and a fast corner for hold.
- **The place-and-route timer is not the signoff timer.** Innovus's internal engine is good enough to optimise against, but signoff exports the netlist with extracted parasitics and re-times in a dedicated tool, expecting the two to disagree slightly.
- **verify_drc is not physical verification.** It checks routing rules inside the tool. Real DRC and LVS run against a foundry runset in a separate verification tool. Nangate45 is an academic library with no foundry behind it, so this does not apply here, but it would immediately on real silicon.
- **Whole categories are absent.** IR drop and electromigration, antenna fixing, metal fill, scan insertion and chain reordering, and formal equivalence checking between the RTL and the synthesised netlist.
- **Flows are normally inherited, not authored.** Companies have methodology teams that own these scripts, and a block designer supplies the RTL, the constraints and a configuration file of block-specific knobs. Writing one from scratch was worth doing once for the understanding.

6. Conclusion

The five-stage pipelined RISC-V CPU reaches a routed layout on Nangate45 and meets setup timing at a 3.00 ns clock, closing at 0.009 ns of setup slack across 4,321 standard cells and 79,481 microns of copper. The flow that produced it is scripted end to end, so the result is reproducible from the repository alone.

Two things are worth carrying forward. The first is that placement, not routing, did almost all of the timing work, which says the netlist handed over by synthesis was physically reasonable and that the remaining margin will not be found by tuning the back end. The second is that the critical path is dominated by a single ripple-carry chain in the ALU while the area is dominated by the register file, so the change that buys speed and the change that buys area are different changes to different parts of the design.

The number this run cannot supply is the maximum achievable frequency, because an optimiser that meets its constraint stops there. Establishing it means sweeping the clock period until the design fails, and that sweep is what the per-run directory structure and the generated results table exist to support.

Appendix A. Reproducing This Run

The run is fully described by the scripts in the repository. Every report quoted above is committed under results/ so each number can be checked against its source.

```
./run.sh --period 3.0 --name 00-baseline --note "..."

runs/00-baseline/reports/     the reports this document parses
results/00-baseline/reports/  the same files, committed as evidence
results/QOR.md        one row per run, for comparing iterations
```

This document was generated from 00-baseline by scripts/make_report.py. The numbers are parsed from the run's reports, not transcribed, so regenerating it after a rerun cannot produce a document that disagrees with its own evidence. Repository: estaresinic05/Silicon-From-Scratch.